

non-functional-requirements

Non-functional requirements

Functional requirements say what Revix does. These say how well it has to do it, and they are the requirements that decide whether anybody can actually use the thing on the day.

This document works through the nine standard categories. For each one it says what the requirement is for a project of this shape, where in the code it is met, how we know, and what is still missing. Where something is not implemented it says so plainly, with the reason, because a requirement we consciously declined is a different thing from one we forgot.

Every measurement quoted here is one we took, and the command that produces it is given so it can be taken again.

Summary

#	Requirement	State	Where
1	Performance	Met, measured	Write/read split, gzip, cache headers, <code>X-Response-Time-ms</code>
2	Scalability	Met for the stated load	Pagination caps, bounded pool, nightly batch
3	Portability	Met	Docker, uv lockfile, env-only config, no host assumptions
4	Usability	Met, audited	WCAG 2.1 AA clean on all ten pages, enforced in CI
5	Compatibility	Met	Generated API client, responsive layout, standard browsers
6	Security	Met at this threat model	Headers, CORS allow-list, rate limit, read-only public API, authenticated admin, no secrets in repo
7	Reliability	Met	Degrades per source, bounded timeouts, no traceback ever leaves
8	Maintainability	Met	Strict types, 14 test modules, ADRs, generated client
9	Availability	Partly met, honestly	503 health contract, keep-warm; single instance is the known limit

1. Performance

Requirement. A verdict page answers in under 300 ms at the 95th percentile, and stays there while the corpus grows.

How it is met. Not by optimisation. By the architectural decision in proposal section 8: the write path and the read path are strictly separated, and no model runs during a user request. Fusion, scoring, bootstrap intervals and claim generation all happen in the nightly pipeline and land in the `serving` schema as finished rows. Every endpoint is one

indexed read of a row that already exists. There is nothing on the read path that can get slower as the corpus grows, because the corpus is not on the read path.

That decision is load-bearing for four other things at once, which is why it is worth naming: it also removes inference cost per request, removes the rate-limit exposure of a third-party model, and removes the possibility of a live demo failing because somebody else's API is down.

Three things were added on top of it:

- **Compression.** `GZipMiddleware` at a 500-byte floor. The full catalogue response is 55.3 kB uncompressed and 5.9 kB gzipped, 9.3 times smaller. That is the response a phone on a weak connection waits on. Small bodies are left alone, because compressing 40 bytes spends CPU to save nothing.
- **Cache directives.** `Cache-Control: public, max-age=300, stale-while-revalidate=1200` on every read endpoint, `no-store` on `/health`. Safe only because of the write/read split: these rows are written once a night and never by a request, so a cached response cannot be stale relative to something the user just did. `stale-while-revalidate` is the part that earns its place on a free tier, where it lets a client show a slightly old copy instantly while refreshing behind it, instead of watching a cold start.
- **A measurable claim.** Every response carries `X-Response-Time-ms`. The 300 ms target used to be checkable only by a benchmark we ran on ourselves; now anyone can read it off a response.

Measured. Endpoints answer in 22 to 35 ms warm locally, and 96 to 247 ms from India against the deployed instance, inside the 300 ms target end to end including the network. The compare page renders in 0.35 s warm.

Known limit. A cold start on Render's free tier is several seconds, which is a platform property and not a code path.

`keepwarm.yml` pings the instance on a schedule to keep it out of that state; see Availability.

2. Scalability

Requirement. Growth in the catalogue or the corpus must not degrade the service, and one caller must not be able to consume it.

How it is met.

- **Reads do not scale with the corpus at all.** A verdict is one row. Going from 1,235 evidence units to 100,000 changes how long the nightly pipeline takes and changes nothing about a request.
- **Every list endpoint is paginated with a hard ceiling.** `/variants` takes `limit` (default 50, maximum 200) and `offset`, validated by FastAPI before the handler runs, so no caller can ask for the whole table. `/metrics` is clamped the same way.
- **The connection pool is bounded**, with overflow and a 10 s wait ceiling. Under a burst, requests queue briefly and then fail fast rather than piling up behind browsers that have already given up.
- **The expensive work is batch and off the request path.** The nightly run takes 53m28s inside a 120-minute timeout, and nobody is waiting on it.
- **Rate limiting** bounds what one client can take: 120 requests per minute per address, sliding window.

Known limit, stated rather than hidden. The rate limiter holds its counters in process memory. Two instances would each allow the full rate, so it bounds what one instance absorbs rather than enforcing a global quota. That is the correct shape for the actual risk here, which is one scraper walking every variant id until the connection pool is exhausted, not a distributed attack. A global quota needs Redis, and adding a second network dependency to the read path in order to protect the read path is a bad trade at this size. If Revix ever ran more than one instance, this is the first thing that would need replacing.

3. Portability

Requirement. The system runs the same on a developer laptop, in CI, and in production, on any of the three operating systems the team uses.

How it is met.

- **The API ships as a Docker image**, multi-stage, 364 MB, running as a non-root user. Nothing in it depends on the host.
- **Dependencies are locked.** A uv workspace with a single `uv.lock` across all three Python packages, so CI and a laptop resolve to identical versions.
- **All configuration is environment variables**, read through one Pydantic `Settings` class with typed defaults. There is no configuration file to copy, no path hard-coded to anybody's machine, and no code that behaves differently because of where it is running. Changing the database, the CORS origins, the rate limit or the evidence floor is an environment change.
- **Postgres is the only external requirement**, and the same DSN drives the app, the CLI and Alembic.
- **Developed on Windows, built and tested on Linux in CI, deployed to Linux.** That is the portability claim tested continuously rather than asserted.

4. Usability

Requirement. A person choosing a vehicle can reach an answer and see what it rests on, including a person using a screen reader or a keyboard.

How it is met.

- **Accessibility is audited, not assumed.** axe-core runs against all ten pages under WCAG 2.1 AA and CI fails on a violation, the operations console included: a page only we use is still a page somebody has to use. `npm run ally`. The colour palette was reworked once specifically because contrast failed, and the audit is what caught it.
- **Semantic HTML and real routes.** Every page is server-rendered with a URL worth sharing; the evidence drawer is its own page rather than a modal, so a citation can be linked to.
- **The interface answers in words.** Divergence is shown as "broad agreement", "some disagreement" or "sharply split" rather than 0.61, and the number stays available underneath for anyone who wants it.

- **Uncertainty is visible.** Verdicts carry the evidence count, the effective sample size and their sources, and a verdict below the evidence floor is suppressed and says so rather than being quietly shown as fact.
- **Failure states exist.** `error.tsx`, `loading.tsx` and `not-found.tsx` at the app root, so a dead API produces a message rather than a blank page.
- **The client never hangs.** An 8-second fetch timeout, added after a host that accepted the connection and then stalled hung a build indefinitely.

5. Compatibility

Requirement. The frontend and backend cannot drift apart, and the site works across current browsers and screen sizes.

How it is met.

- **The API contract is generated, not written twice.** The OpenAPI schema is the source of truth and the TypeScript client is generated from it with `openapi-typescript`. A response shape that changes without the frontend changing is a type error at build time, not a runtime surprise. This is the strongest compatibility guarantee in the project and it is structural.
- **Standards, not vendor features.** Plain JSON over HTTP, schema.org JSON-LD for structured data, no browser-specific API anywhere.
- **Responsive layout** via Tailwind breakpoints, verified down to phone widths.
- **The browser matrix is whatever Next.js targets**, which is current Chrome, Firefox, Safari and Edge. E2e runs headless Chromium in CI.
- **CORS is explicit**, listing both loopback spellings, because a browser treats `localhost` and `127.0.0.1` as different origins. Getting this wrong produced a healthy 200 in the server log and a broken type-ahead in the browser, which is exactly the failure this category exists to catch.

6. Security

Requirement. Revix holds no user accounts and no personal data, so the threat model is narrow and worth stating precisely: protect the credentials that reach our sources, do not become a vector against the reader's browser, and do not let one caller degrade the service for everybody.

How it is met.

- **The public API is read-only.** Every endpoint outside `/admin` is a GET and CORS permits only GET, so for every reader of the site the entire class of injection-through-mutation vulnerabilities has no surface.
- **There is exactly one write, and it is authenticated.** The operations console records a person's decision about which vehicle a listing refers to. It is one nullable foreign key on one row, behind the gate described below. This qualifies the sentence above rather than contradicting it, and it is written here rather than left as a stale claim that used to be true.

- **No raw SQL.** Everything goes through SQLAlchemy's typed ORM with bound parameters, including the free-text search, which is the one place user input reaches a query.
- **Secrets are never in the repository**, which is public. `.env` is gitignored, credentials live in GitHub Secrets and Render's dashboard, and CI runs a hygiene check that fails the build on a committed secret. The `revix probe` command exists so a key can be verified by whoever holds it without anybody having to send it.
- **Security headers on the web app.** Content-Security-Policy, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy` declining camera, microphone and location, and HSTS for two years. Vercel adds none of these by default, so before this the site shipped with a sniffable content type, no framing rule, and a full referrer travelling to every outbound link, and every outbound link here goes to a review on somebody else's site.
- **The same headers on the API**, applied by middleware so they are present on errors and refusals too, not only on the happy path.
- **CORS is an allow-list**, never a wildcard, with credentials disabled.
- **Rate limiting**, 120 requests per minute per client, sliding window, with `Retry-After` and the remaining budget published so a well-behaved client can slow down before it is refused. `/health` is exempt, because rate limiting a health check means taking the instance down in order to protect it.
- **No traceback ever reaches a caller.** A catch-all returns a fixed JSON shape carrying a request id and logs the exception server-side. The test for this asserts that a filesystem path raised inside a handler does not appear in the response body.
- **The container runs as a non-root user** on an unprivileged uid.
- **Our own outbound behaviour is part of this too.** The connectors obey `robots.txt`, rate limit themselves, and identify honestly. Sources that refuse us are left alone rather than worked around.

One trade-off, made deliberately. The CSP allows `'unsafe-inline'` for scripts. Next's App Router bootstraps hydration from inline script tags, and the alternative is a per-request nonce, which opts every page out of static rendering. That would spend the one non-functional requirement with a number attached to it, latency, to defend a surface this app does not have: no authentication, no cookie or token worth stealing, no user-generated content, and the only user text that reaches a page is the search box, which React escapes. It is written down in `next.config.ts` so it stays a decision rather than becoming an oversight.

Authentication, on the one surface that needs it. The operations console is gated by HTTP Basic over HTTPS, both halves compared in constant time and both always compared, so the response does not leak whether a username exists. It **fails closed**: with the credentials unset every admin route answers 503 and none of them touch the database, because a console that behaves identically whether or not it is protected is one nobody notices is unprotected. The browser sends the operator's own credentials rather than our server sending credentials on a visitor's behalf, and they travel in a header we set explicitly rather than a cookie, so no CSRF surface exists on the write. See [ADR 0010](#).

Still not implemented, and why. No password reset, no second account, no roles. There is one operator and one authorisation question, and a role system guarding a single boolean is machinery pretending to be security. A second principal of any kind is the trigger to revisit, and the first thing it would need is an audit trail rather than roles.

7. Reliability

Requirement. A failure in one part must not become a failure of the whole, and the system must tell the truth about what state it is in.

How it is met.

- **Sources degrade, they do not break.** A connector that fails records the failure on its ingest run and the pipeline continues with the others. `/sources/health` publishes where every source stands, so a dead source is visible rather than silently absent. Team-BHP returning 403 and two sites disallowing us in `robots.txt` are all visible states, not crashes.
- **A demo does not depend on anything remote.** Because no model runs on the read path, the site keeps working when a third-party API is down, over quota, or slow. This was the explicit reason for the architecture.
- **Every wait is bounded.** Database connect timeout 5 s, pool wait 10 s, pool recycle 240 s ahead of Neon dropping idle connections, plus `pool_pre_ping`. This one was a real gap found while writing this document: `/health` is written to report an unreachable database as a 503, but without a connect timeout it never got the chance, because psycopg waited on the operating system's TCP timeout while the platform's probe gave up first. A hang is a worse failure than an error, and it was turning a correct 503 into no answer at all.
- **The health contract is honest.** `/health` opens its own session rather than taking the dependency, precisely so a database failure is a value the handler can report instead of an exception thrown before the handler runs. It answers 503 with `database: false`, which is what a platform probe reads.
- **The pipeline is idempotent and re-runnable.** Ingest is content-hashed and deduplicated, so a partial run can be repeated without double-counting.
- **The container stops cleanly.** `CMD` uses exec form so unicorn actually receives SIGTERM, which was wrong once and meant in-flight requests were killed on every deploy.
- **Every request is identifiable.** A request id on every response, echoed from the caller if supplied, and quoted in error bodies, so a report of "it broke" becomes a single grep.

8. Maintainability

Requirement. Three people work on this at once, and it has to survive a handover to whoever reads it next.

How it is met.

- **Types are enforced, not decorative.** mypy strict across all three Python packages, TypeScript strict on the frontend, SQLAlchemy 2.0 typed ORM, Pydantic v2 at the boundaries.
- **One linter and formatter, run in CI.** ruff for both.
- **Tests.** 14 test modules covering connectors, fusion, extraction, schema, the API contract, CORS, health, and now the non-functional guarantees themselves, plus Playwright e2e and an accessibility audit. The suite has caught real defects: a macro-F1 averaged over zero-support aspects, a connector recording our own search term as the source's answer, an unordered LIMIT that made ingest and fusion disagree about which variant they meant.

- **Decisions are written down.** ADRs for the choices that would otherwise look arbitrary later, including why we use official APIs only, why the fusion experiment is built the way it is, and why the source floor is two rather than three.
- **Generated code where duplication would rot.** The API client is generated from the schema, so the frontend cannot drift from the backend by hand.
- **Configuration is one class,** with every knob defaulted and documented in place, so behaviour is changed in one file rather than hunted for.
- **Separation by lifecycle.** Four Postgres schemas, `raw`, `core`, `analysis`, `serving`, so it is always obvious which layer a table belongs to and what may write to it.

9. Availability

Requirement. The site is up when somebody looks at it, and specifically on 16 October.

How it is met.

- **The read path has one dependency,** Postgres. No model, no third-party API, nothing that has to be reachable for a page to render.
- `keepwarm.yml` pings the instance on a schedule so Render's free tier does not idle it into a cold start.
- **The health endpoint is a real contract,** 503 when the database is unreachable, so the platform stops routing to an instance that cannot serve instead of sending traffic to one that will fail.
- **Static assets are on Vercel's CDN,** so the frontend stays up independently of the API, and with the cache directives above a browser holding a recent response can render while the API wakes.
- **Failures are contained.** A nightly pipeline failure leaves yesterday's verdicts serving; the site does not go down because an ingest run did.

Known limit, stated rather than hidden. This is a single instance on a free tier. There is no redundancy, no failover, and no multi-region deployment, so the honest availability claim is "resilient to its dependencies failing", not "highly available". Buying that would mean paying for at least two instances and a load balancer, which is outside what this project is. The mitigations above target the failure mode that actually threatens a demo, a cold start or a sleeping database, rather than the one that does not, a datacentre losing power.

What this pass changed

Six of the nine categories were already met by decisions made earlier, mostly by the write/read split, which quietly satisfies performance, scalability, reliability and availability at once. The audit found and fixed:

1. **No security headers anywhere.** Neither the web app nor the API sent any. Now both do, including on errors.
2. **No rate limiting.** One caller could exhaust the connection pool.
3. **No compression or cache directives,** so every client re-fetched 55 kB of rows that change once a night.

4. **Unbounded database waits.** The most serious of the four, because it defeated the `/health` 503 contract that was already written and tested: the handler could not report an unreachable database because it never regained control to do so.
5. **No request identity and no request logging.** A failure report could not be tied to a request.

Reproducing the measurements

```
uv run pytest tests/test_nfr.py -q          # the guarantees, asserted
cd apps/web && npm run ally                 # WCAG 2.1 AA, all ten pages
curl -sI $API/variants | grep -Ei 'cache-control|x-response-time|x-ratelimit'
curl -s -H 'Accept-Encoding: gzip' -o /dev/null -w '%{size_download}\n' $API/variants?
    limit=200
curl -s -H 'Accept-Encoding: identity' -o /dev/null -w '%{size_download}\n' $API/variants?
    limit=200
```